Section 13 / 30

Regular Expressions

Regular expressions (`RegEx`) are like the art of crafting precise blueprints for searching patterns in text or files. They allow you to find, replace, and manipulate data with incredible precision. Think of RegEx as a highly customizable filter that lets you sift through strings of text, looking for exactly what you need—whether it's analyzing data, validating input, or performing advanced search operations.

At its core, a regular expression is a sequence of characters and symbols that together form a search pattern. These patterns often involve special symbols called metacharacters, which define the structure of the search rather than representing literal text. For example, metacharacters allow you to specify whether you're searching for digits, letters, or any character that fits a certain pattern.

RegEx is available in many programming languages and tools, such as grep or sed, making it a versatile and powerful tool in a our toolkit.

Grouping

Among other things, regex offers us the possibility to group the desired search patterns. Basically, regex follows three different concepts, which are distinguished by the three different brackets:

Grouping Operators

	Operators	Description
1	`(a)`	The round brackets are used to group parts of a regex. Within the brackets, you can define further patterns which should be processed together.
2	`[a-z]`	The square brackets are used to define character classes. Inside the brackets, you can specify a list of characters to search for.
3	`{1,10}`	The curly brackets are used to define quantifiers. Inside the brackets, you can specify a number or a range that indicates how often a previous pattern should be repeated.
4	`|`	Also called the OR operator and shows results when one of the two expressions matches
5	`.*`	Operates similarly to an AND operator by displaying results only when both expressions are present and match in the specified order

Suppose we use the `OR` operator. The regex searches for one of the given search parameters. In the next example, we search for lines containing the word `my` or `false`. To use these operators, you need to apply the extended regex using the `-E` option in grep.

OR operator

```shellsession
cry0l1t3@htb:~$ grep -E "(my|false)" /etc/passwd

lxd:x:105:65534::/var/lib/lxd/:/bin/false
pollinate:x:109:1::/var/cache/pollinate:/bin/false
mysql:x:116:120:MySQL Server,,:/nonexistent:/bin/false
```

Since one of the two search parameters always occurs in the three lines, all three lines are displayed accordingly. However, if we use the `AND` operator, we will get a different result for the same search parameters.

AND operator

```shellsession
cry0l1t3@htb:~$ grep -E "(my.*false)" /etc/passwd

mysql:x:116:120:MySQL Server,,:/nonexistent:/bin/false
```

Basically, what we are saying with this command is that we are looking for a line where we want to see both `my` and `false`. A simplified example would also be to use `grep` twice and look like this:

```shellsession
cry0l1t3@htb:~$ grep -E "my" /etc/passwd | grep -E "false"

mysql:x:116:120:MySQL Server,,:/nonexistent:/bin/false
```

Here are some optional tasks to help you practice RegEx and improve your ability to handle them more effectively. These exercises will use the `/etc/ssh/sshd_config` file on your `Pwnbox` instance, allowing you to explore real-world applications of RegEx in a configuration file. By completing these tasks, you'll gain hands-on experience in working with patterns, searching, and manipulating text in practical scenarios.

1	Show all lines that do not contain the `#` character.
2	Search for all lines that contain a word that starts with `Permit`.
3	Search for all lines that contain a word ending with `Authentication`.
4	Search for all lines containing the word `Key`.
5	Search for all lines beginning with `Password` and containing `yes`.
6	Search for all lines that end with `yes`.

Connect to HTB

13 / 30 Sections

Mark Complete & Next